

Exercise 3 — Important Concepts of React.js

5 small programs, written as simply as possible for first-time React learners.

How to Run Any Program Below

Step	Do this
1	Install Node.js from nodejs.org (LTS version)
2	Open terminal and run: <code>npm create vite@latest my-app -- --template react</code>
3	<code>cd my-app</code> then <code>npm install</code>
4	Open src/App.jsx , delete everything, paste the program
5	Run <code>npm run dev</code> and open the link shown (localhost:5173)

Note: If your lab uses Create React App, paste the code into **src/App.js** — the code is the same. Program (b) needs **internet**.

5 Words You Must Know

Word	Simple meaning
useState	Stores data that can change. When it changes, the page updates
useEffect	Runs some code after the page shows (e.g. loading data)
props	Data a parent component passes to a child component
map()	Loops over a list and makes one element per item
key	A unique id React needs for each item in a list

a) Counter using useState Hook

AIM

To write a React program to implement a counter button using the React useState hook.

PROGRAM (App.jsx)

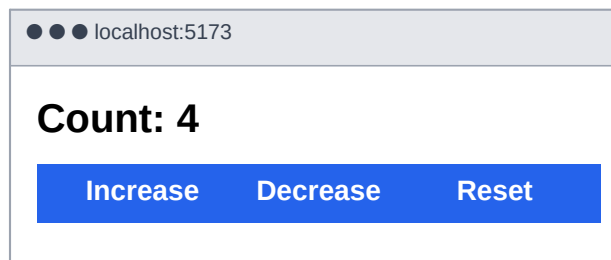
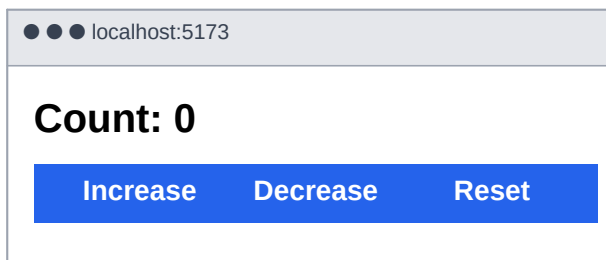
```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <button onClick={() => setCount(count - 1)}>Decrease</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default App;
```

OUTPUT



HOW IT WORKS



KEY POINTS

- **useState(0)** → count starts at 0.
- **setCount(count + 1)** → add 1, **setCount(count - 1)** → subtract 1.
- **setCount(0)** → back to 0. Every change updates the page automatically.

b) Fetch Data from an API using useEffect Hook

AIM

To write a React program to fetch data from an API using the React useEffect hook.

PROGRAM (App.jsx)

```
import { useState, useEffect } from "react";

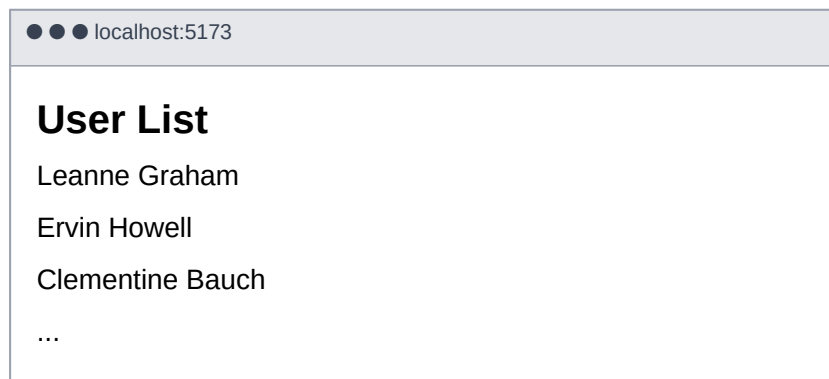
function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => setUsers(data));
  }, []);

  return (
    <div>
      <h1>User List</h1>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

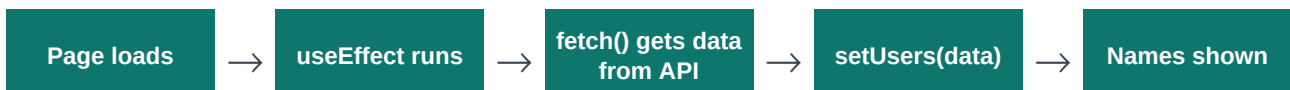
export default App;
```

OUTPUT



Output (shows 10 names — first 3 shown here)

HOW IT WORKS



KEY POINTS

- **useState([])** → users starts as an empty list.
- **fetch(url)** → asks the API for data. **res.json()** → converts it to JavaScript.
- **[]** at the end of **useEffect** → run only **once**, when the page first loads.

c) Two Components Sharing Data using Props

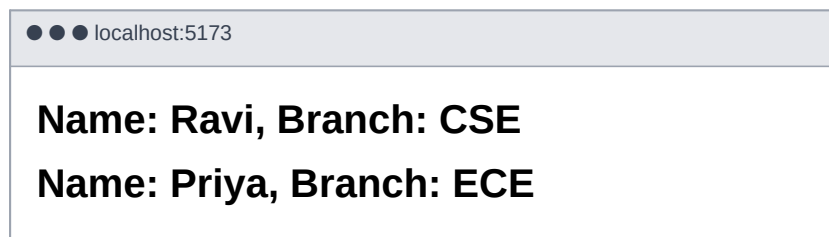
AIM

To write a React program with two React components sharing data using props.

PROGRAM (App.jsx)

```
function Student(props) {  
  return <h2>Name: {props.name}, Branch: {props.branch}</h2>;  
}  
  
function App() {  
  return (  
    <div>  
      <Student name="Ravi" branch="CSE" />  
      <Student name="Priya" branch="ECE" />  
    </div>  
  );  
}  
  
export default App;
```

OUTPUT



Output in browser

HOW DATA FLOWS



KEY POINTS

- **App** is the parent, **Student** is the child — these are the two components.
- Parent sends data like HTML attributes: **name="Ravi"**.
- Child reads it using **props.name**. Same component, different data!

d) Forms in React

AIM

To write a React program to implement forms in React.

PROGRAM (App.jsx)

```
import { useState } from "react";

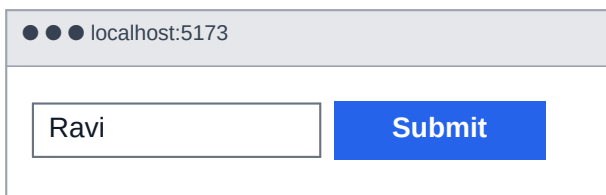
function App() {
  const [name, setName] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    alert("Hello " + name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  );
}

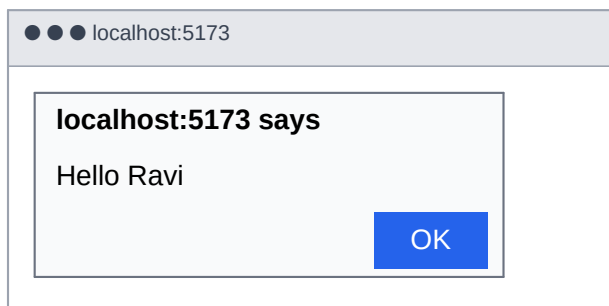
export default App;
```

OUTPUT



● ● ● localhost:5173

Type a name



● ● ● localhost:5173

localhost:5173 says
Hello Ravi

After clicking Submit

KEY POINTS

- **value={name}** → the input box shows what is stored in state.
- **onChange** → every time you type, **e.target.value** (the typed text) is saved.
- **e.preventDefault()** → stops the page from refreshing on submit.

e) Iterative Rendering using map()

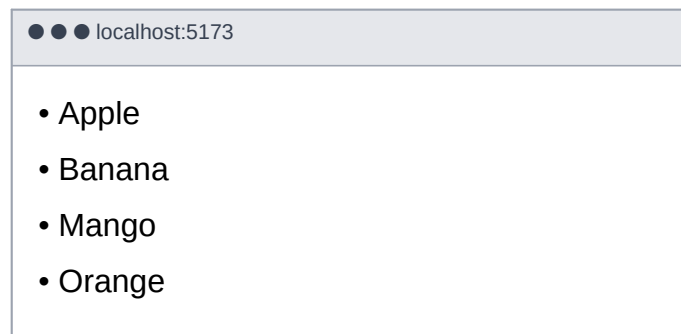
AIM

To write a React program to implement iterative rendering using the map() function.

PROGRAM (App.jsx)

```
function App() {  
  const fruits = ["Apple", "Banana", "Mango", "Orange"];  
  
  return (  
    <ul>  
      {fruits.map((fruit, index) => (  
        <li key={index}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default App;
```

OUTPUT



Output in browser

HOW map() WORKS

["Apple", "Banana", "Mango", "Orange"]



map()



4 x items on page

KEY POINTS

- **map()** goes through the list one item at a time.
- For each fruit, it creates one **** — so 4 fruits = 4 list items.
- **key** gives each item a unique id (React shows a warning without it).

RESULT

All five React programs were written and executed successfully, and the outputs were verified in the browser.